

Project: Robust Anomaly Detection in Noisy Time-Series Data

DDL: 24 May 2026

1 Introduction

This project focuses on anomaly detection in **noisy time-series data**, a problem commonly encountered in real-world applications such as financial markets and monitoring systems. In such scenarios, observations are often **highly contaminated by noise**, while anomalous events are rare, leading to **severe class imbalance**.

You are given sequential data derived from financial market observations, where each time step is associated with a label indicating whether it corresponds to anomalous behavior. The goal is to design machine learning models that can accurately identify anomalies from these noisy temporal signals. Unlike standard classification problems, anomalies in time-series data are often characterized by **temporal patterns** rather than isolated point-wise deviations, requiring models to effectively utilize sequential information.

To evaluate both effectiveness and robustness, the project consists of two tasks. The first task focuses on anomaly detection under a given data distribution. The second task considers a more complex scenario, where the underlying data characteristics and anomaly patterns differ from those observed in the first task, making detection significantly more challenging. This setting reflects practical situations in which models must handle evolving and more intricate forms of anomalous behavior.

Overall, the project aims to assess the ability to build accurate and robust machine learning solutions for anomaly detection in noisy time-series data. Students work in teams of up to 4 people.

1.1 Problem setting

You are given a **time-series anomaly detection** dataset with the following characteristics:

- **Noisy observations:** the data are highly stochastic and contaminated by noise
- **Severe class imbalance:** anomalous events are rare
- **Temporal dependency:** anomalies are often characterized by patterns over time rather than isolated point-wise deviations

Concretely, the dataset consists of a sequence of observations:

$$(x_t, y_t)_{t=1}^T, \tag{1}$$

where $x_t \in \mathbb{R}^d$ is the feature vector at time step t , $y_t \in \{0, 1\}$ indicates whether the observation is anomalous, and T denotes the number of time steps in the dataset. The data are ordered in time and should be treated as a continuous temporal sequence.

Your goal is to design learning models that:

1. Accurately detect anomalies from noisy and imbalanced time-series data by effectively leveraging temporal information
2. Remain robust under more complex data scenarios

1.2 Data overview

You will be provided with:

- **train.csv** – labeled training data

- Columns: $f_1, f_2, \dots, f_d, y$
- You may use this file (and only this labeled file) to train, validate, and select your models.
- **test_simple.csv** – unlabeled test data under a similar distribution
 - Same feature columns as `train.csv`, but without the label y .
 - Used for **Task 1** evaluation.
- **test_complex.csv** – unlabeled test data under a more complex scenario
 - Feature columns are compatible with the training data (same dimension and ordering).
 - Used for **Task 2** evaluation.

The true labels for both test files are hidden and will be used by the instructor for grading.

2 Tasks

2.1 Task 1: Anomaly Detection under the Given Distribution

Using only the labeled training data (`train.csv`), you must:

1. Train a model to predict the anomaly label y_t for each time step
2. Produce predictions for **every row** in `test_simple.csv`

You will submit a prediction file:

- `pred_simple.csv`, containing a **single** column named `y_pred`
- The first row must be the header: `y_pred`
- Each subsequent row should contain one predicted label (0 or 1)
- The i -th entry of `y_pred` must correspond to the i -th row in `test_simple.csv`
- The number of rows must be exactly the same as `test_simple.csv`

Notes:

- The dataset is highly imbalanced, which should be taken into account when training and evaluating models.
- The data are time-ordered, so validation should respect temporal dependencies. You may construct inputs using multiple time steps, but the temporal order must be preserved.
- You are expected to design appropriate validation procedures to select and tune your models.

2.2 Task 2: Robust Anomaly Detection in Complex Scenarios

In this task, you must:

1. Apply the same trained model from Task 1 to a new dataset `test_complex.csv`.
2. Generate anomaly predictions for every row in `test_complex.csv`.

You will submit a prediction file:

- `pred_complex.csv`, containing a single column named `y_pred`
- The first row must be the header: `y_pred`
- Each subsequent row should contain one predicted label (0 or 1)
- The i -th entry of `y_pred` must correspond to the i -th row in `test_complex.csv`
- The number of rows must be exactly the same as in `test_complex.csv`

Notes:

- You are not allowed to retrain, fine-tune, or adapt your model from Task 1. The performance in this task will depend on how well your model generalizes beyond the training data, so model design in Task 1 is important.
- No labels are provided for this dataset, but the feature format is the same as in Task 1 with a more complex anomaly pattern.

3 Grading Scheme

Each team receives a final project score based on two components:

$$\text{FinalScore} = \min(100, \alpha \cdot (\text{ReportScore} + \text{AlgoPerformanceScore})),$$

where

- $\text{ReportScore} \in [0, 30]$,
- $\text{AlgoPerformanceScore} \in [0, 70]$,
- α is a scaling factor that depends on the team size.

3.1 Team-size-dependent scaling factor α

Let m denote the number of students in the team. The scaling factor α is defined as:

$$\alpha = \begin{cases} 1.1, & \text{if } m = 2 \\ 1.0, & \text{if } m = 3 \\ 0.9, & \text{if } m = 4 \end{cases}$$

The final score is capped at 100. That is, if $\alpha \cdot (\text{ReportScore} + \text{AlgoPerformanceScore}) > 100$, the final score is recorded as 100.

3.2 Report component (up to 30 points)

The report should be clearly structured and written in understandable academic English. In particular, it should:

- Briefly situate your work in the context of related methods for time-series anomaly detection.
- Explain how your algorithm is designed and implemented for Task 1 and Task 2, including key modeling design, how temporal information is utilized, and how challenges such as noise and class imbalance are addressed.
- Present your validation strategy, including the evaluation metrics used, and how it is applied for model selection and tuning.
- Provide experimental results on validation or held-out data, and analyze the behavior of your model, including its potential strengths and limitations under more complex scenarios.
- Clearly describe the division of work among team members.

3.3 Algorithm performance component (up to 70 points)

The algorithm performance score ($\text{AlgoPerformanceScore}$) is based on predictive performance on both Task 1 and Task 2.

For each task, performance is evaluated using metrics selected by the instructor (e.g., AUC-PR, F-score). Teams that meet a minimum performance threshold receive a base score, with additional points awarded based on relative ranking across all teams and both tasks.

Higher scores are assigned to teams that achieve strong performance on Task 1 and maintain robustness under the more complex scenario in Task 2.

4 Submissions

Each team should make one joint submission; all team members share the same set of files and final scores. You must submit the following files:

1. **Report**

- TeamID_Name_report.pdf
- Example: 12_SanZhang_report.pdf, where Name is the group leader's name.

2. **Predictions and model**

- TeamID_Name_pred_simple.csv
- TeamID_Name_pred_complex.csv
- TeamID_Name_model.xxx (trained model file, e.g. .pkl or .pt)

3. **Code**

- TeamID_Name_code.zip containing your source code and configuration files.

All files should be submitted before the deadline announced on bb.

5 Prohibition and Academic Integrity

You will receive **0 marks** for this project if any of the following occur:

- Late submission beyond the grace period specified by the instructor.
- Plagiarism in the report or code (copying from other students, online repositories).
- Label leakage or cheating, including but not limited to:
 - Directly searching for the hidden labels of the test sets.
 - Using external labeled data that are too similar to the test sets without explicit permission.
 - Manually tuning predictions after seeing any part of the hidden labels.

You may reuse known algorithms and open-source implementations (e.g., from scikit-learn), but you must cite your sources and clearly explain how you used or modified them.